

Universidad de Costa Rica

Sede de Occidente - Recinto de Grecia

Curso: Estructuras de Datos

Informe del Proyecto Programado

Sistema de administración de libros de biblioteca

Integrantes: Marcos Ferreto Estrada — C5F092

Paulo Anchía Correás — [Carné]

Profesora: [Nombre de la profesora]

Grupo: [Número de grupo]

Fecha: June 21, 2026

Índice

1	Introducción	2
2	Descripción del problema	2
3	Análisis del problema	2
3.1	Estructuras de datos utilizadas	2
3.1.1	Árbol AVL para libros	2
3.1.2	Tabla hash para estudiantes	3
3.1.3	Árbol rojinegro para préstamos	3
3.2	Carga y persistencia de archivos	3
3.3	Resolución de subproblemas	4
3.4	Funciones o métodos principales	5
4	Análisis de resultados	6
5	Conclusiones y recomendaciones	7
6	Bibliografía	7

1 Introducción

Este documento describe el desarrollo del proyecto programado para administrar los datos de la biblioteca de la Universidad de Costa Rica, Recinto de Grecia. El sistema permite cargar, almacenar, consultar, eliminar y prestar libros mediante el uso de estructuras de datos estudiadas en el curso.

2 Descripción del problema

El problema consiste en diseñar e implementar una aplicación de software que gestione información de libros, estudiantes y préstamos de una biblioteca utilizando estructuras de datos eficientes. El sistema permite buscar, insertar, eliminar y registrar préstamos, proporcionando una solución integral a las necesidades de la biblioteca.

Todos los datos se persisten en archivos de texto plano delimitados (con extensión `.xml`) y, al ejecutar el programa, se cargan en las estructuras de datos apropiadas en memoria principal. El sistema maneja libros identificados por un código único, estudiantes por su carné y préstamos con un código autogenerated. Además, el programa incluye validaciones de negocio estrictas, impidiendo operaciones inválidas como prestar un libro que ya se encuentra prestado, o eliminar a un estudiante que mantenga devoluciones pendientes. Todo esto se apoya en una Interfaz Gráfica de Usuario (GUI) construida con `tkinter` para facilitar la interacción visual y la usabilidad del programa final.

3 Análisis del problema

3.1 Estructuras de datos utilizadas

3.1.1 Árbol AVL para libros

Los libros se almacenan en un árbol AVL ordenado por su código numérico único. Esta estructura de datos resulta idónea debido a que mantiene el balance automáticamente tras cada inserción o eliminación, garantizando un tiempo de complejidad de $O(\log n)$ en búsquedas, lo cual es vital cuando el catálogo de libros crece.

Datos almacenados por libro:

- Código único.
- Autor.
- Título.
- Año de publicación.
- Editorial.

- Área o temática.

3.1.2 Tabla hash para estudiantes

Los estudiantes se gestionan a través de una tabla hash. La función hash se calcula a partir del número de carné del estudiante utilizando el método de división o módulo sobre el tamaño de la tabla. Las colisiones se resuelven de forma eficiente utilizando el encadenamiento a través de listas simples enlazadas. Esta estructura reduce el tiempo de acceso promedio para ubicar el perfil de un estudiante.

Datos almacenados por estudiante:

- Carné único.
- Nombre.
- Carrera que cursa.
- Teléfono.
- Correo electrónico.
- Dirección física.

3.1.3 Árbol rojinegro para préstamos

Los registros de préstamos de libros se indexan en un Árbol Rojinegro. Al tratarse de operaciones transaccionales donde las inserciones y las consultas son frecuentes, esta estructura maneja muy bien el balance sin generar rotaciones excesivas (a diferencia del AVL). Mantiene los tiempos de operación acotados en $O(\log n)$ sin degradarse ante la inserción de fechas u órdenes consecutivos.

Datos almacenados por préstamo:

- Código único de préstamo.
- Código del libro prestado.
- Carné del estudiante solicitante.
- Fecha de préstamo.
- Fecha de devolución esperada.

3.2 Carga y persistencia de archivos

El programa cuenta con un módulo **XMLManager** especializado en la lectura de los archivos de texto `libros.xml`, `estudiantes.xml` y `prestamos.xml`, que pese a su nombre, almacenan atributos delimitados por el carácter de porcentaje (%). Al iniciar la aplicación, se abren estos archivos y cada línea se separa para instanciar directamente los objetos **Libro**, **Estudiante** y **Prestamo**. Posteriormente, al momento de registrar un nuevo préstamo o al ejecutar una eliminación de un estudiante o libro válido, el programa recorre la estructura de datos correspondiente (haciendo uso

del recorrido inorden o listando el hash) y sobrescribe los archivos, asegurando la persistencia de los cambios.

3.3 Resolución de subproblemas

En la siguiente tabla se detalla brevemente cómo se abordó la implementación de cada requerimiento o subproblema planteado en el proyecto.

Subproblema	Solución implementada
Buscar libros por autor	Se realiza un recorrido inorden sobre el árbol AVL, evaluando si el autor del nodo actual coincide o contiene la subcadena de búsqueda. Se retorna una lista de coincidencias.
Buscar libro por título	Similar a la búsqueda por autor, un recorrido completo sobre el AVL compara el título en cada nodo, ignorando mayúsculas/minúsculas para encontrar todas las similitudes.
Buscar libro por código	Se aprovecha la propiedad de ordenamiento del árbol AVL. Se evalúa recursivamente si el código buscado es menor (izq) o mayor (der) al actual, reduciendo el tiempo de acceso a $O(\log n)$.
Eliminar libro por código	Primero se hace una verificación en el Árbol Rojinegro para asegurar que el libro no exista en ningún préstamo activo. Si está prestado, se rechaza. Si está disponible, se procede con la eliminación del nodo AVL y re-balanceo.
Buscar estudiante por carné	Se calcula el hash del carné para hallar el índice en el arreglo de la tabla hash. Posteriormente, se busca en la lista enlazada ubicada en esa posición hasta encontrar el carné exacto.
Buscar estudiante por nombre	Se iteran todas las listas enlazadas que integran la tabla hash buscando coincidencias totales o parciales con la cadena proporcionada.
Buscar estudiantes por carrera	Sigue un procedimiento similar al de búsqueda por nombre, recorriendo la tabla hash e insertando los registros coincidentes en una lista de resultados.
Eliminar estudiante por carné	Antes de eliminar, el sistema valida que el estudiante no tenga ningún código de préstamo asociado en el Árbol Rojinegro. De lo contrario, impide la eliminación. Si es válida, elimina el nodo de la lista enlazada en el hash correspondiente.

Realizar préstamo	Se verifica primero la existencia del estudiante y del libro. Luego se corrobora que el libro no esté ya prestado. Al cumplirse las validaciones, se instancia el préstamo asignándole una duración de 15 días, se inserta en el Árbol Rojinegro y se actualiza el archivo <code>prestamos.xml</code> .
Mostrar árboles en inorden	Se utilizan métodos recursivos <code>inorden(nodo)</code> que visitan el hijo izquierdo, procesan la raíz y visitan el hijo derecho para asegurar el orden ascendente por código (AVL y Rojinegro).
Interfaz gráfica	Se programó usando el módulo estándar <code>tkinter</code> . Se distribuyó en pestañas (<code>Notebook</code>) para dividir claramente las funciones de Libros, Estudiantes y Préstamos. Cuenta con formularios de ingreso <code>Entry</code> , <code>Button</code> explícitos y un <code>Treeview</code> para mostrar los datos devueltos en forma tabular de manera limpia e intuitiva.

3.4 Funciones o métodos principales

A continuación, se listan los métodos de integración y gestión más importantes, correspondientes a los controladores del proyecto (`SistemaPrestamos`, `GestorEliminacion` y `XMLManager`).

Función o método	Parámetros	Retorna	Descripción
<code>cargar_todo</code>	Ninguno (lee de los atributos de clase)	Tupla de 3 listas de objetos	Lee las líneas de los archivos <code>.xml</code> , instancia cada clase y retorna los conjuntos en crudo para popular la GUI.
<code>dar_prestamo</code>	<code>codigo_libro</code> , <code>carnet</code>	(<code>bool</code> , <code>str</code>)	Valida y registra un préstamo si el libro está disponible. Retorna una tupla con un valor booleano de éxito y un mensaje de estado.
<code>eliminar_libro</code>	<code>codigo</code>	(<code>bool</code> , <code>str</code>)	Realiza las verificaciones de préstamos activos y elimina el libro del AVL si todo es correcto, luego sincroniza en disco.

<code>eliminar_estudiante</code>	<code>carnet</code>	<code>(bool, str)</code>	Ejecuta validaciones análogas pero dirigidas a la tabla hash del estudiante, también sincronizando al disco en caso exitoso.
----------------------------------	---------------------	--------------------------	--

4 Análisis de resultados

La siguiente tabla resume el grado de completitud técnica de las diferentes etapas y requerimientos del sistema en el momento de la entrega del código final.

Parte del proyecto	Estado	Observaciones
Archivos XML	Terminado con éxito	Uso correcto de la persistencia directa mediante porcentaje (%).
Árbol AVL	Terminado con éxito	Funcional con rebalanceo automático, inserción, búsqueda y eliminación.
Tabla hash	Terminado con éxito	Funcional con resolución de colisiones y extracción de datos.
Árbol rojinegro	Terminado con éxito	Funcional en balanceo por colores para administrar los registros.
Búsquedas	Terminado con éxito	Algoritmos de búsqueda acoplados correctamente a la GUI.
Carga de archivos	Terminado con éxito	Funciona eficientemente para poblar los árboles al iniciar.
Eliminación de datos	Terminado con éxito	Validaciones integradas que impiden corromper los préstamos.
Préstamos	Terminado con éxito	Validación lógica estricta y control de fechas.
Interfaz gráfica de usuario	Terminado con éxito	Pantallas organizadas por pestañas, componentes limpios.
Documentación interna	Terminado con éxito	Funciones documentadas e informe elaborado.

5 Conclusiones y recomendaciones

1. **Conclusión:** El uso adecuado de las estructuras de datos mejoró sustancialmente la eficiencia y escalabilidad del sistema comparado a utilizar simples arreglos secuenciales.
Recomendación: Es recomendable mantener un análisis de complejidad inicial antes de empezar futuros desarrollos para escoger el modelo de datos correcto desde la fase de planeación.
2. **Conclusión:** El Árbol AVL demostró ser altamente útil y óptimo para mantener los códigos de libros ordenados y responder rápidamente a búsquedas exactas.
Recomendación: En implementaciones donde las eliminaciones de libros sean abrumadoramente masivas, el re-balanceo constante podría penalizar el rendimiento, por lo que podría recomendarse estudiar alternativas de baja frecuencia en ese caso hipotético.
3. **Conclusión:** La tabla hash permitió el acceso directo y ágil al registro de los estudiantes mediante su número de carné, optimizando significativamente la operación.
Recomendación: Para bases de datos universitarias reales masivas, se recomienda calibrar y testear el tamaño de la tabla (actualmente 100) frente al uso esperado para no castigar el factor de carga ni generar listas de colisión excesivamente largas.
4. **Conclusión:** El manejo del Árbol Rojinegro brindó la estabilidad necesaria para las operaciones de préstamos, las cuales usualmente implican un crecimiento secuencial del identificador.
Recomendación: Aprovechar el ordenamiento del árbol rojinegro para en una futura iteración añadir reportes automáticos de libros próximos a vencer su fecha de devolución.
5. **Conclusión:** La programación de las validaciones de dependencia previas (comprobar si hay un préstamo antes de permitir borrar un libro) previno proactivamente las inconsistencias en disco e interrupciones del programa.
Recomendación: Mantener una lógica fuerte del lado del controlador de persistencia antes de alterar memoria, para que sirva de segunda capa de validación.
6. **Conclusión:** La Interfaz Gráfica de Usuario (GUI) construida con los componentes nativos, le dio un aspecto intuitivo y organizado a una lógica subyacente que de otra forma sería compleja de probar por comandos.
Recomendación: Se puede ampliar el sistema incluyendo filtros avanzados por fechas, reportes PDF o alertas visuales cuando un estudiante acumula penalizaciones y morosidades, lo cual complementaría excelentemente a la GUI construida.

6 Bibliografía

- Python Software Foundation. (2026). *tkinter — Interfaz gráfica de usuario*. Documentación oficial de Python. Recuperado de <https://docs.python.org/es/3/library/tkinter.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Material didáctico y diapositivas de la cátedra del curso "Estructuras de Datos", Universidad de Costa Rica, Recinto de Grecia.